

AI PRINCIPLES & TECHNIQUES

FOUR IN A ROW

MARIA THIELE
ANGELINA PODOŁAKO
RADBOD UNIVERSITY

s1130739
s1125886
October 11, 2024

Contents

1	Introduction	1
2	Methods	2
2.1	Specification	2
2.2	Design	2
2.3	Implementation	3
3	Testing	5
4	Results	6
5	Discussion	7
6	Conclusion and Reflection	8
6.1	Main findings	8
6.2	Limitations and further research	8
6.3	Work division and remarks	8
	References	8
A	MiniMax algorithm	9
B	Alpha-Beta Pruning	10
C	Visualization of Results	12

1 Introduction

In two player games such as chess or tic-tac-toe winning the game is not only dependent on the moves of oneself but also on the moves of the opponent (Codecademy, 2024). To maximize the chance of winning, each player needs to take into account the possible moves the opponent can play. An optimal move, given any game state, would be one that maximizes the chance of winning while at the same time minimizes the opponents chance of winning. To determine such an optimal move algorithms like the MiniMax algorithm can be used. In short, the MiniMax algorithm explores all possible moves and the resulting game outcomes and determines which move would most likely guarantee the player to win.

Since games like chess have many possible biard states, building the entire game tree takes too much time and space and is therefore not feasible. To solve this problem,

the MiniMax algorithm is only applied to a certain depth of the game tree. To further decrease the complexity, approaches like alpha-beta pruning can be applied. Alpha-beta pruning is an approach that uses the MiniMax algorithm but does not evaluate branches that are not going to be chosen based on the evaluation of previous branches.

To test the difference in complexity between using the MiniMax algorithm and alpha-beta pruning, both approaches are compared in a game of "Four in a Row".

2 Methods

2.1 Specification

The program first initiates a board of Four in a Row with two players. The players can either be human players or computer players which use the MiniMax algorithm or alpha-beta pruning. The board has a width of 7 columns and a height of 6. In order to win, 4 in a row is needed. Those parameters are the default setting but they can be manually changed in the code. After the board is initialized, the two players take turns by choosing a column to play in. The game can end in a draw or with one player winning. The total number of board evaluations is given for each player. Those evaluations will form the basis of comparing the runtime complexity of the MiniMax algorithm and alpha-beta pruning.

2.2 Design

The game is programmed in an object orientated way using different classes to separate functionality. The classes can be found in 4 separate files.

In the file app.py the game is initialized and started. In board.py, the class Board can be found, which handles the creation of a board, playing on the board and making a humanly readable presentation of the board. To evaluate a board state, a heuristic is needed. The responsible Heuristic class can be found in the file heuristics.py and includes an abstract class for a heuristic, as well as the class SimpleHeuristic that inherits from the abstract class. Finally, the file players.py handles the different type of players. The abstract class PlayerController is inherited by three other classes: HumanPlayer, MinMaxPlayer and AlphaBetaPlayer. As the names indicate, a player from the class MinMaxPlayer calculates the optimal column to play in by using the Minimax algorithm, while a player from the class AlphaBetaPlayer uses alpha-beta pruning to determine the optimal column to play in.

The pseudo code below shows the main idea behind the MiniMax algorithm. It is taken from the website GeeksforGeeks (2024).

```
1 MiniMax Algorithm Pseudocode:
2
3 def minmax(state, depth, maximizing_player):
4     if is_terminal(state) or depth == 0:
5         return utility(state)
6
7     if maximizing_player:
8         max_eval = -infinity
9         for action in actions(state):
10             eval = minmax(result(state, action), depth - 1, False)
11             max_eval = max(max_eval, eval)
12     return max_eval
13 else:
```

```

14         min_eval = infinity
15         for action in actions(state):
16             eval = minmax(result(state, action), depth - 1, True)
17             min_eval = min(min_eval, eval)
18         return min_eval

```

The pseudo code for alpha beta pruning is based on the MinMax algorithm. The pseudocode displayed below was copied from GeeksforGeeks (2023).

```

1 Alpha-Beta Pruning Pseudocode:
2
3 function minimax(node, depth, isMaximizingPlayer, alpha, beta):
4
5     if node is a leaf node :
6         return value of the node
7
8     if isMaximizingPlayer :
9         bestVal = -INFINITY
10        for each child node :
11            value = minimax(node, depth+1, false, alpha, beta)
12            bestVal = max( bestVal, value)
13            alpha = max( alpha, bestVal)
14            if beta <= alpha:
15                break
16        return bestVal
17
18    else :
19        bestVal = +INFINITY
20        for each child node :
21            value = minimax(node, depth+1, true, alpha, beta)
22            bestVal = min( bestVal, value)
23            beta = min( beta, bestVal)
24            if beta <= alpha:
25                break
26        return bestVal

```

2.3 Implementation

If one or both players of the game are human players, a column is suggested based on the heuristic. However, the human player can freely choose between the columns as long as the chosen column is valid, meaning the column exists and the column is not yet full. If one or both players are either MiniMax players or alpha-beta pruning players, the algorithm tries to determine the best column and plays in that column. The algorithm uses backtracking to determine the best possible move for the player. The nodes of the lowest depth are evaluated first. Based on whose players turn it is (the max or the min player), the nodes from higher levels are evaluated. When all nodes are evaluated, the path with the best evaluation is chosen.

Alpha-beta pruning works very similar but in this version of the MiniMax algorithm, not all branches of the tree need to be evaluated. By taking the evaluations of other branches into account, it can be avoided to go through a branch that will not be chosen because another branch already has a more promising evaluation.

The code for the Minimax algorithm is shown below.

```

1 def minmax(self, board: Board, depth: int, player: int):
2     """Minmax recursive algorithm for evaluating game states.
3
4     Args:

```

```

5         board (Board): Current board state.
6         depth (int): Depth of search.
7         player (int): The current player (either 1 or 2).
8
9     Returns:
10         tuple: (score, best_column)
11     """
12     # Determining the ID of the opponent based on the current player
13     if player == 1:
14         opponent = 2
15     else:
16         opponent = 1
17
18     # Base case
19     if depth == 0 or board.is_valid == False or self.heuristic.winning
20     (board.board_state, self.game_n) != 0:
21         return self.heuristic.evaluate_board(player, board), None
22
23     # Recursive Call
24     # Player is Maximizing player
25     if player == self.player_id:
26         max_eval = -np.inf
27
28         for col in range(board.width):
29             if board.is_valid(col):
30                 new_board = board.get_new_board(col, player)
31
32                 evaluation, _ = self.minmax(new_board, depth - 1,
33                 opponent)
34
35                 # updating the max score
36                 if evaluation > max_eval:
37                     max_eval = evaluation
38                     best_column = col
39
40         return max_eval, best_column
41
42     # Player is Minimizing Player
43     else:
44         min_eval = np.inf
45
46         for col in range(board.width):
47             if board.is_valid(col):
48                 new_board = board.get_new_board(col, player)
49
50                 evaluation, _ = self.minmax(new_board, depth - 1,
51                 opponent)
52
53                 # updating the min score
54                 if evaluation < min_eval:
55                     min_eval = evaluation
56                     best_column = col
57
58         return min_eval, best_column

```

The code is quite similar to the pseudo code given in section 2.2. However, slight modifications were needed regarding the return value of the function.

Since the algorithm is supposed to identify the best column to play in, another return argument was added. The variable `best_column` stores the column with the best evaluation score. This is done in lines 35-37 for the maximizing player and in lines 52-54 for the minimizing player. Another slight change is the `player` parameter of the function. While the pseudo code uses a bool variable to identify the player, an integer value is

used in the code. This is explained by the fact that each player is given an integer id, which makes it easier to reference this id instead of first converting it into a bool variable. The opponent is determined based on the current player in lines 13-16. The code for implementing alpha-beta pruning does not differ much from the MiniMax algorithm. Before returning max_val and best_column (or min_val and best_column), the variables alpha (for the max player) and beta (for the min player) are updated. The code for updating alpha in the max player part is shown below.

```
1      # updating alpha
2      alpha = max(alpha, evaluation)
3      # Pruning
4      if beta <= alpha: # Beta cut-off
5          break
```

The entire code for the algorithms can be found in sections A and B.

3 Testing

In order to compare the runtime complexity of the MiniMax algorithm and alpha-beta pruning, the number of necessary board state evaluations is counted. This information is given at the end of each game for each player. There are 4 parameters that can be changed

- The width of the board
- The height of the board
- The number n in a row required to win
- The depth to which the nodes are evaluated

The heuristics used for evaluation has been provided and will remain constant for testing. When comparing the two algorithms, the game should be played with one MiniMax player and one alpha-beta player. The parameters above are varied one at a time for each game. In total, each parameter has at least three different values while the other parameters stay constant. The width of the board is either 7, 8, 10 or 20. The height of the board is either 6, 7, 10 or 20. The number of the same symbols in a row varies between 4 and 6. Lastly, the depth of the search tree takes a value from 2 to 5. In addition to the number of evaluations, the outcome of the game is reported.

4 Results

Each parameter of section 3 has been varied and the results are summarized in Table 1.

	MiniMax	Alpha-Beta	width	height	n in a row	result
Depth: 2	482	171	7	6	4	Draw
	482	169	7	6	5	AlphaBeta won
	514	186	7	6	6	Draw
	688	218	8	6	4	Draw
	1261	331	10	6	4	Draw
	8969	1330	20	6	4	AlphaBeta won
	526	189	7	7	4	MiniMax won
	762	269	7	10	4	Draw
	1462	514	7	20	4	Draw
Depth: 3	2401	617	7	6	4	AlphaBeta won
	2374	613	7	6	5	Draw
	2359	613	7	6	6	Draw
	3972	876	8	6	4	Draw
	9268	1568	10	6	4	Draw
	135042	10287	20	6	4	MiniMax won
	2747	695	7	7	4	AlphaBeta won
	3965	1033	7	10	4	AlphaBeta won
	7875	2073	7	20	4	AlphaBeta won
Depth: 4	16150	1030	7	6	4	Draw
	16207	934	7	6	5	AlphaBeta won
	16239	936	7	6	6	Draw
	30176	1459	8	6	4	Draw
	86044	2612	10	6	4	AlphaBeta won
	2323304	17955	20	6	4	Draw
	17950	1187	7	7	4	Draw
	25085	1742	7	10	4	Draw
	47320	3482	7	20	4	Draw
Depth: 5	86164	5046	7	6	4	AlphaBeta won
	85969	5122	7	6	5	Draw
	85898	5133	7	6	6	Draw
	184768	7951	8	6	4	Draw
	665631	17013	10	6	4	Draw
	37542386	190319	20	6	4	Draw
	101624	6012	7	7	4	AlphaBeta won
	143240	8830	7	10	4	AlphaBeta won
	285935	18290	7	20	4	AlphaBeta won

Table 1: Comparison between MiniMax and alpha-beta pruning

A visualization of those findings is shown in figure 1. Note that this figure does not include every data point because some data points were so large that the rest of the data points was hardly differentiable. The code used for this visualization can be found in appendix C.

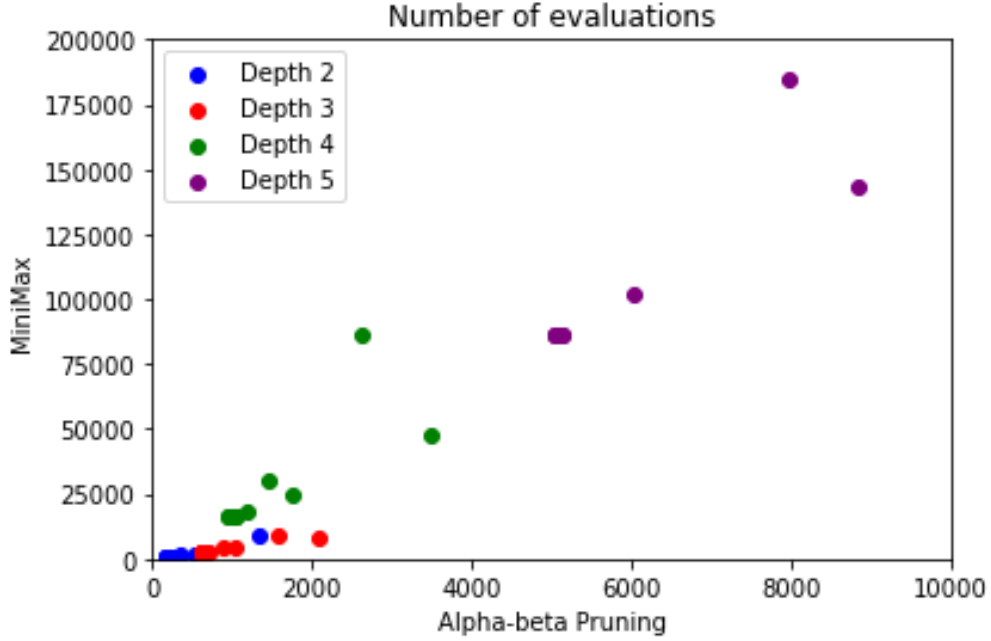


Figure 1: Visualization of the number of evaluations of MiniMax vs Alpha-beta pruning

5 Discussion

As can be seen from the results, alpha-beta pruning is a lot faster than the MiniMax algorithm. In every scenario tested, the number of evaluations using alpha-beta pruning is lower than the amount of evaluations using the MiniMax algorithm. This is reasonable, since alpha-beta pruning is based on the MiniMax algorithm but does not necessarily need to evaluate every branch if the evaluation of previous branches already determines that another branch will be used.

Especially when the board has a lot of columns, the complexity varies a lot. For the depth of 4 and 20 columns, MiniMax took about 129 times longer than alpha-beta pruning. For a depth of 5, it was about 200 times longer.

Nonetheless, increasing the depths of the search also leads to a visible increase in the amount of evaluations for both algorithms. This is not surprising, considering that another level of depths also increases the amount of board states that can be reached from the root node. While all parameters change the complexity of the algorithms, the width of the board seems to have the most influence. For every column added to the board, another branch is added in the search tree. This leads to a fast accumulation of nodes and therefore more evaluations. Even though the height of the board also changes the complexity, its influence is not as big as the width of the board. Basically, a higher board allows columns to be valid for a longer time because more moves can be played in the column. The overall search tree increases in levels and therefore also the evaluations increase.

It is interesting to see that while most games end in a draw, the alpha-beta algorithm wins more games than the MiniMax algorithm. However, the data does not provide an explanation as to why. One assumption could be that the ordering of player1 and player2 is important. In all test cases, player1 used the MiniMax algorithm while player2 used alpha-beta pruning.

6 Conclusion and Reflection

6.1 Main findings

The results show that alpha-beta pruning has a significantly lower runtime complexity in comparison to the MiniMax algorithm. The largest difference was that the Minimax algorithm took about 200 times longer than the alpha-beta pruning. The difference is not always that much, however, there always is a difference. Therefore, alpha-beta pruning is a more efficient way to do game search.

6.2 Limitations and further research

One possible limitation of this research is the fact that for each test player1 and player2 did not switch. Player1 has always used the MiniMax algorithm while player2 always used alpha-beta pruning. It can be assumed that player1 automatically does more evaluations than player2 because player1 starts the game and has therefore got more options to choose from. The extend to which this plays a role should be further researched.

In addition, some games ended because one player won. Since the game ended before the board was completely full, the number of evaluations is likely going to be smaller than if the game had ended in a draw. This should not change the fact that alpha-beta pruning is more efficient, but it should also not be overlooked.

Future research could also focus on the effects of the parameters that can vary. It would be interesting to see how each parameter effects the complexity and if the effects on the complexity vary depending on the algorithm.

6.3 Work division and remarks

For comparing the complexity of the two algorithms, a game of Four in a Row was needed. The framework of this game was provided by Kwisthout (2024).

The two algorithms were implemented by the authors. Both authors contributed both to programming the algorithms and writing the report, however, Angelina did most of the programming while Maria focused mainly on the report.

The assignment specifies the algorithm as MinMax whereas the lecture slides and internet sources call the algorithm MiniMax. Since part of the code has been provided, the naming convention MinMax was kept. In this report, the name MiniMax was used because the slides and the sources used call it MiniMax algorithm. Either way, both names describe the same algorithm.

References

- Codecademy. (2024). *Minimax algorithm*. Retrieved October 11, 2024, from <https://www.codecademy.com/resources/docs/ai/minimax-algorithm>
- GeeksforGeeks. (2023). *Minimax algorithm in game theory — set 4 (alpha-beta pruning)*. Retrieved October 11, 2024, from <https://www.geeksforgeeks.org/minimax-algorithm-in-game-theory-set-4-alpha-beta-pruning/>
- GeeksforGeeks. (2024). *Mini-max algorithm in artificial intelligence*. Retrieved October 11, 2024, from <https://www.geeksforgeeks.org/mini-max-algorithm-in-artificial-intelligence/>
- Kwisthout, J. H. P. (2024). *Aipt assignments*. GitLab. <https://gitlab.socsci.ru.nl/j.kwisthout/aipt-assignments>

A MiniMax algorithm

```
1 class MinMaxPlayer(PlayerController):
2     """Class for the minmax player using the minmax algorithm
3     Inherits from Playercontroller
4     """
5     def __init__(self, player_id: int, game_n: int, depth: int, heuristic:
6         Heuristic) -> None:
7         """
8         Args:
9             player_id (int): id of a player, can take values 1 or 2 (0 =
10             empty)
11             game_n (int): n in a row required to win
12             depth (int): the max search depth
13             heuristic (Heuristic): heuristic used by the player
14         """
15         super().__init__(player_id, game_n, heuristic)
16         self.depth: int = depth
17
18     def make_move(self, board: Board) -> int:
19         """Gets the column for the player to play in using the Minmax
20         algorithm.
21
22         Args:
23             board (Board): the current board
24
25         Returns:
26             int: column to play in
27         """
28         # Use minimax to get the best move
29         _, best_column = self.minimax(board, self.depth, self.player_id)
30
31         # Return the best move
32         return best_column
33
34     def minimax(self, board: Board, depth: int, player: int):
35         """Minmax recursive algorithm for evaluating game states.
36
37         Args:
38             board (Board): Current board state.
39             depth (int): Depth of search.
40             player (int): The current player (either 1 or 2).
41
42         Returns:
43             tuple: (score, best_column)
44         """
45         # Determining the ID of the opponent based on the current player
46         if player == 1:
47             opponent = 2
48         else:
49             opponent = 1
50
51         # Base case
52         if depth == 0 or board.is_valid == False or self.heuristic.winning
53         (board.board_state, self.game_n) != 0:
54             return self.heuristic.evaluate_board(player, board), None
55
56         # Recursive Call
57
58         # Player is Maximizing player
59         if player == self.player_id:
60             max_eval = -np.inf
```

```

60
61         for col in range(board.width):
62             if board.is_valid(col):
63                 new_board = board.get_new_board(col, player)
64
65                 evaluation, _ = self.minmax(new_board, depth - 1,
66 opponent)
67
68                 # updating the max score
69                 if evaluation > max_eval:
70                     max_eval = evaluation
71                     best_column = col
72
73             return max_eval, best_column
74
75         # Player is Minimizing Player
76         else:
77             min_eval = np.inf
78
79             for col in range(board.width):
80                 if board.is_valid(col):
81                     new_board = board.get_new_board(col, player)
82
83                     evaluation, _ = self.minmax(new_board, depth - 1,
84 opponent)
85
86                     # updating the min score
87                     if evaluation < min_eval:
88                         min_eval = evaluation
89                         best_column = col
90
91             return min_eval, best_column

```

B Alpha-Beta Pruning

```

1 class AlphaBetaPlayer(PlayerController):
2     """Class for the minmax player using the minmax algorithm with alpha-
3     beta pruning
4     Inherits from Playercontroller
5     """
6     def __init__(self, player_id: int, game_n: int, depth: int, heuristic:
7         Heuristic) -> None:
8         """
9         Args:
10             player_id (int): id of a player, can take values 1 or 2 (0 =
11             empty)
12             game_n (int): n in a row required to win
13             depth (int): the max search depth
14             heuristic (Heuristic): heuristic used by the player
15         """
16         super().__init__(player_id, game_n, heuristic)
17         self.depth: int = depth
18
19     def minmax_abp(self, board: Board, depth: int, alpha: float, beta:
20         float, player: int):
21         """Minimax algorithm with alpha-beta pruning to evaluate the best
22         move
23
24         Args:
25             board (Board): the current board
26             depth (int): current depth in tree

```

```

23         alpha (float): the best score that the maximizing player can
guarantee
24         beta (float): the best score that the minimizing player can
guarantee
25         player (int): The current player (either 1 or 2).
26
27     Returns:
28         tuple: (score, best_column)
29     """
30     if player == 1:
31         opponent = 2
32     else:
33         opponent = 1
34
35     # Base case
36     if depth == 0 or board.is_valid == False or self.heuristic.winning
(board.board_state, self.game.n) != 0:
37         return self.heuristic.evaluate_board(player, board), None
38
39     # Recursive Call
40
41     # Player is Maximizing player
42     if player == self.player_id:
43         max_eval = -np.inf
44
45         for col in range(board.width):
46             if board.is_valid(col):
47                 new_board = board.get_new_board(col, player)
48
49                 evaluation, _ = self.minmax_abp(new_board, depth - 1,
alpha, beta, opponent)
50
51                 # updating the max score
52                 if evaluation > max_eval:
53                     max_eval = evaluation
54                     best_column = col
55
56                 # updating alpha
57                 alpha = max(alpha, evaluation)
58                 # Pruning
59                 if beta <= alpha: # Beta cut-off
60                     break
61
62         return max_eval, best_column
63
64     # Player is Minimizing Player
65     else:
66         min_eval = np.inf
67
68         for col in range(board.width):
69             if board.is_valid(col):
70                 new_board = board.get_new_board(col, player)
71
72                 evaluation, _ = self.minmax_abp(new_board, depth - 1,
alpha, beta, opponent)
73
74                 # updating the min score
75                 if evaluation < min_eval:
76                     min_eval = evaluation
77                     best_column = col
78
79                 # updating beta
80                 beta = min(beta, evaluation)
81                 # Pruning
82                 if beta <= alpha: # Alpha cut-off
83                     break

```

```

84         return min_eval, best_column
85
86
87     def make_move(self, board: Board) -> int:
88         """Gets the column for the player to play in
89
90         Args:
91             board (Board): the current board
92
93         Returns:
94             int: column to play in
95         """
96
97         # TODO: implement minmax algorithm with alpha beta pruning!
98         alpha = -np.inf
99         beta = np.inf
100
101         _, best_column = self.minmax_abp(board, self.depth, alpha, beta,
102                                         self.player_id)
103
104         return best_column

```

C Visualization of Results

```

1  #!/usr/bin/env python3
2  # -*- coding: utf-8 -*-
3  """
4  Created on Fri Oct 11 22:32:29 2024
5
6  @author: maria
7  """
8
9  import matplotlib.pyplot as plt
10
11  # depth 2
12  # minmax data
13  d2_w_mm = [482, 688, 1261, 8969]
14  d2_h_mm = [482, 526, 762, 1462]
15  d2_n_mm = [482, 482, 514]
16  d2_mm = d2_w_mm + d2_h_mm + d2_n_mm
17
18  # alpha beta data
19  d2_w_ab = [171, 218, 331, 1330]
20  d2_h_ab = [171, 189, 269, 514]
21  d2_n_ab = [171, 169, 186]
22  d2_ab = d2_w_ab + d2_h_ab + d2_n_ab
23
24  # depth 3
25  # minmax data
26  d3_w_mm = [2401, 3972, 9268, 135042]
27  d3_h_mm = [2401, 2747, 3965, 7875]
28  d3_n_mm = [2401, 2374, 2359]
29  d3_mm = d3_w_mm + d3_h_mm + d3_n_mm
30
31  # alpha beta data
32  d3_w_ab = [617, 876, 1568, 10287]
33  d3_h_ab = [617, 695, 1033, 2073]
34  d3_n_ab = [617, 613, 613]
35  d3_ab = d3_w_ab + d3_h_ab + d3_n_ab
36
37  # depth 4
38  # minmax data
39  d4_w_mm = [16150, 30176, 86044, 2323304]

```

```

40 d4_h_mm = [16150, 17950, 25085, 47320]
41 d4_n_mm = [16150, 16207, 16239]
42 d4_mm = d4_w_mm + d4_h_mm + d4_n_mm
43
44 # alpha beta data
45 d4_w_ab = [1030, 1459, 2612, 17955]
46 d4_h_ab = [1030, 1187, 1742, 3482]
47 d4_n_ab = [1030, 934, 936]
48 d4_ab = d4_w_ab + d4_h_ab + d4_n_ab
49
50 # depth 5
51 # minmax data
52 d5_w_mm = [86164, 184768, 665631, 37542386]
53 d5_h_mm = [86164, 101624, 143240, 285935]
54 d5_n_mm = [86164, 85969, 85898]
55 d5_mm = d5_w_mm + d5_h_mm + d5_n_mm
56
57 # alpha beta data
58 d5_w_ab = [5046, 7951, 17013, 190319]
59 d5_h_ab = [5046, 6012, 8830, 18290]
60 d5_n_ab = [5046, 5122, 5133]
61 d5_ab = d5_w_ab + d5_h_ab + d5_n_ab
62
63
64 plt.figure()
65 plt.scatter(d2_ab, d2_mm, color = 'blue', label = 'Depth 2')
66 plt.scatter(d3_ab, d3_mm, color = 'red', label = 'Depth 3')
67 plt.scatter(d4_ab, d4_mm, color = 'green', label = 'Depth 4')
68 plt.scatter(d5_ab, d5_mm, color = 'purple', label = 'Depth 5')
69 plt.xlabel('Alpha-beta Pruning')
70 plt.ylabel('MiniMax')
71 plt.title('Number of evaluations')
72 plt.xlim(0, 10000)
73 plt.ylim(0, 200000)
74 plt.legend()
75 plt.show()

```